TECHSERVICE HUB

PRD Addendum v1.1

Wallet-Based Payment System

Version	1.1 — supersedes Section 5.4 and parts of Sections 6, 7, 8, 9 of PRD v1.0
Date	November 1, 2026
Status	PROPOSED — PENDING FOUNDER SIGN-OFF
Authors	Founder A (Engineering) · Founder B (Design & Docs)
Reference	ManagementEwallet reference screens

Table of Contents

1. Summary of the Change
2. Goals & Success Metrics
3. Reference Screens Analysed
4. Scope
5. User Roles & Permissions
6. Functional Requirements (6.1 Wallet · 6.2 Deposits · 6.3 Wallet Checkout · 6.4 Refunds · 6.5 Withdrawals · 6.6 Ledger · 6.7 Admin Panel · 6.8 Notifications)
7. Core Flows
8. Order Lifecycle Impact
9. Data Model Changes
10. API Changes
11. Page Map Changes
12. UX Requirements
13. Security, Fraud & Compliance
14. Non-Functional Requirements
15. Migration Plan
16. Delivery Phases
17. Acceptance Criteria (Key Scenarios)
18. Analytics & KPIs to Instrument
19. Open Questions for Founders
20. Changes to PRD v1.0 (Traceability)

1. Summary of the Change

PRD v1.0 uses a per-order manual payment: the customer pays outside the platform and submits a transaction ID against each order, and an admin verifies it. v1.1 replaces this with a wallet-based system, based on the ManagementEwallet reference screens:

1. The customer **deposits** money into a personal TSH Wallet (bKash, Nagad, Rocket or Binance) by sending money to the platform's number/address, then submitting the amount + Transaction ID.
2. An admin **approves or declines** the deposit. On approval the wallet is credited.
3. At checkout the customer **pays for the order from wallet balance** instantly. No per-order TrxID submission.
4. The customer can request a **withdrawal/refund** of unused balance, which an admin reviews and pays out.
5. Every money movement is recorded in an **immutable ledger** shown in the customer's History page.

Why: one-time verification per deposit instead of per order, faster checkout, simpler refunds (credit back to wallet), clean accounting, and a path to automated gateways later.

2. Goals & Success Metrics

Goal	Metric	Target (launch + 90 days)
Faster checkout	Time from "Place order" to "Paid" for users with sufficient balance	< 5 seconds
Fast deposit confirmation	Median time deposit submitted → approved/declined	< 1 hour
Reduce admin workload	Payment verifications per paid order	≤ 0.6 (down from 1.0)
Financial accuracy	Wallet balance = sum of ledger entries (reconciliation mismatches)	0
Fraud control	Duplicate/fake TrxID deposits credited	0
Adoption	Share of paid orders funded from wallet	≥ 90%

3. Reference Screens Analysed

Customer — "Deposit" page (sidebar: Overview, Deposit, Withdraw, History, Profile)

- Method tabs: **bKash / Nagad / Binance**.
- Dashed box showing the platform receiving number (e.g. 01XXX-XXXXXX) with a **Copy** button.

- 3-step instructions: copy number → send from your bKash/Nagad/Binance app → confirm with amount and Transaction ID.
- Fields: **Amount (৳)** and **Transaction ID**; button **Confirm**.

Admin panel

- KPI cards: Total Users, Total User Balances, Pending Deposits, Pending Withdrawals.
- Tabs: **Deposits, Withdrawals, Users, Settings**.
- Deposit table: Time (UTC), User, Method, Amount, TrxID, Action (Approve / Decline, or an Approved badge once processed).

> **Gaps in the reference to fix in TSH** (all specified below): no decline reason, no duplicate-TrxID protection, no Binance currency conversion, no min/max limits, no ledger or reconciliation, no audit trail, no rejected/expired status badges, no screenshot proof, and the amount field has no validation rules.

4. Scope

In scope (v1.1)

- TSH Wallet per user (single currency: BDT).
- Deposits via bKash, Nagad, Rocket and Binance (USDT) with manual admin verification.
- Pay for orders from wallet; wallet-funded checkout.
- Withdrawals (refund payout) with admin processing.
- Immutable wallet ledger, customer History page, CSV export.
- Admin: deposits queue, withdrawals queue, user wallets, payment Settings, reconciliation report.
- Notifications for every wallet event.
- Migration of existing per-order payments.

Out of scope (v1.1)

- Automated gateway APIs (bKash/Nagad/Binance Pay) — design keeps provider/providerRef hooks.
- Wallet-to-wallet transfers between customers.
- Multi-currency balances (Binance deposits are converted to BDT on credit).
- Interest, cashback or promo credit (reserved for a later release).
- Mixed payment at checkout (part wallet / part external). Customer must top up the shortfall first.

5. User Roles & Permissions

Capability	CUSTOMER	ADMIN
View own wallet balance, history	Yes	Yes (any user)
Submit deposit request	Yes	Yes (own)
Approve / decline deposit	No	Yes
Pay order from wallet	Yes (own orders)	No (cannot spend a customer's balance)
Submit withdrawal request	Yes	Yes (own)
Approve / reject / mark paid withdrawal	No	Yes
Manual balance adjustment (credit/debit)	No	Yes — requires reason, audited, max per-action limit
Edit payment methods, limits, fees	No	Yes

Roles remain server-authoritative; no wallet endpoint accepts a user ID, role, balance or status from the client.

6. Functional Requirements

Priority: MUST / SHOULD / COULD. These requirements replace FR-16, FR-17 and FR-18 and extend FR-24, FR-26, FR-29.

6.1 Wallet

ID	Requirement	Priority
WAL-01	Every user gets exactly one wallet, created automatically at account provisioning (`/auth/sync`). Existing users are back-filled with a zero balance.	**MUST**
WAL-02	Balance is stored as `Decimal(12,2)` in BDT and serialized as a string (no floats).	**MUST**
WAL-03	Balance can never go negative. Enforced by a database `CHECK` constraint and by application logic.	**MUST**
WAL-04	Balance is changed only by creating a ledger entry inside the same database transaction (see 6.6). No code path may update the balance directly.	**MUST**
WAL-05	Wallet shows **Available balance** and **Pending (held) balance** (funds reserved by pending withdrawals).	**MUST**
WAL-06	Wallet has status `ACTIVE` , `FROZEN` , `CLOSED` . A frozen wallet blocks spend and withdrawals but still allows viewing. Admin can freeze with a reason.	**MUST**

6.2 Deposits

ID	Requirement	Priority
DEP-01	Customer selects a method (bKash, Nagad, Rocket, Binance). Only methods enabled in Settings are shown.	**MUST**
DEP-02	The page shows the platform's receiving account for the chosen method (number or wallet address, account type e.g. Personal/Agent, QR for Binance) with a one-tap **Copy** button and a copied confirmation.	**MUST**
DEP-03	Customer submits **Amount** and **Transaction ID**. A screenshot of the payment is optional (SHOULD for Binance and for amounts above the configured threshold).	**MUST**
DEP-04	Amount validation: numeric, 2 decimals max, within the method's min/max (defaults: min ৳100, max ৳50,000 per deposit; configurable).	**MUST**
DEP-05	TrxID uniqueness: a TrxID is unique per method across all non-declined deposits (case-insensitive, trimmed). A duplicate is rejected immediately with `DUPLICATE_TRX_ID` .	**MUST**
DEP-06	New deposits are created with status `PENDING` . The customer sees them in History immediately.	**MUST**
DEP-07	Admin sees all pending deposits oldest-first with: time (UTC), user, method, amount, TrxID, screenshot, and the user's prior approval history.	**MUST**

DEP-08	**Approve:** admin may confirm or correct the received amount (if the actual received amount differs, the corrected amount is credited and the difference is recorded). The wallet is credited via a `DEPOSIT` ledger entry.	**MUST**
DEP-09	**Decline:** admin must pick a reason (`TRX_NOT_FOUND` , `AMOUNT_MISMATCH` , `DUPLICATE` , `SUSPECTED_FRAUD` , `OTHER`) and may add a note. The customer sees the reason.	**MUST**
DEP-10	A processed deposit is final. Approve/decline can be applied only once (idempotent; second click returns the current state).	**MUST**
DEP-11	Pending deposits not processed within 72 hours are flagged `STALE` in the admin queue and the customer is told they can contact support. They are not auto-declined.	**SHOULD**
DEP-12	A customer may cancel their own deposit only while it is `PENDING` .	**SHOULD**
DEP-13	Rate limit: maximum 5 pending deposits per user and 10 submissions per hour.	**MUST**
DEP-14	**Binance (USDT):** customer enters the USDT amount; the system shows the BDT equivalent using the admin-configured rate (Settings). The BDT value is frozen on the deposit at submission and credited on approval. The admin may adjust only with a recorded reason.	**MUST**
DEP-15	Optional deposit fee (percentage and/or flat) per method, configured in Settings, shown to the customer before confirming and recorded as a separate `FEE` ledger entry. Default: 0.	**COULD**

6.3 Paying for Orders from Wallet

ID	Requirement	Priority
PAY-01	At checkout the customer sees wallet balance, order total, and remaining balance after payment.	**MUST**
PAY-02	If balance ≥ total, **Pay with Wallet** places the order and debits the wallet in one database transaction (order creation + `ORDER_PAYMENT` ledger entry + payment record `PAID`).	**MUST**
PAY-03	If balance < total, the checkout shows the shortfall and a **Top up ৳X** shortcut that opens the Deposit page pre-filled, then returns to checkout. The cart is preserved.	**MUST**
PAY-04	The idempotency key (FR-13) also protects the wallet debit: a retried request never charges twice.	**MUST**
PAY-05	The charged amount is computed server-side from the order snapshot. The client sends no price.	**MUST**
PAY-06	The customer-facing order page shows payment status `PAID` with the ledger reference and receipt.	**MUST**
PAY-07	Pricing is not tied to payment method. Wallet balance is BDT only.	**MUST**
PAY-08	Optional partial/milestone payment (e.g. 50% upfront, 50% on delivery): the order stores `amountPaid` and `amountDue` ; the second charge happens automatically from wallet when the order reaches `COMPLETED` , or the customer pays manually.	**COULD**

6.4 Refunds & Cancellations

ID	Requirement	Priority
REF-01	When an order is `CANCELLED` or `REJECTED`, the amount paid is automatically credited back to the wallet as a `REFUND` ledger entry.	**MUST**
REF-02	Customer cancellation remains allowed only in SUBMITTED / UNDER_REVIEW (per PRD v1.0 Section 6) and gives a full refund.	**MUST**
REF-03	Admin may issue a full or partial refund on any order with a reason. Capped at `amountPaid − amountAlreadyRefunded`.	**MUST**
REF-04	Refunds go to the wallet only. Cash-out to bKash/Nagad/Binance is done through a withdrawal request.	**MUST**
REF-05	Refund policy text is configurable in Settings and shown at checkout and on the Withdraw page.	**SHOULD**

6.5 Withdrawals

ID	Requirement	Priority
WDR-01	Customer selects a payout method (bKash, Nagad, Rocket, Binance), enters the destination account/address, the amount and (for Binance) the network.	**MUST**
WDR-02	Validation: amount ≤ available balance; within min/max (defaults: min ৳200, max ৳50,000 per request; configurable); destination format check (11-digit Bangladeshi mobile number for bKash/Nagad/Rocket; address format for Binance).	**MUST**
WDR-03	On submission the requested amount is **held**: available balance decreases and pending balance increases through a `WITHDRAWAL_HOLD` ledger entry.	**MUST**
WDR-04	Statuses: `PENDING → APPROVED → PAID`, or `PENDING → REJECTED` (hold released), or customer-cancelled (hold released).	**MUST**
WDR-05	Admin marks a withdrawal **PAID** after sending the money, entering the outgoing TrxID (required) and optional proof. The hold is converted into a final `WITHDRAWAL` ledger entry.	**MUST**
WDR-06	Admin rejection requires a reason; the hold is released with a `WITHDRAWAL_RELEASE` entry.	**MUST**
WDR-07	A configurable processing fee (percentage/flat) and a daily withdrawal cap per user. Default fee 0.	**SHOULD**
WDR-08	Security step: first withdrawal to a **new destination** requires re-authentication (Firebase recent-login or email OTP) and a 24-hour cooling period before it can be approved.	**SHOULD**
WDR-09	Withdrawals are blocked if the user has an open dispute, a frozen wallet, or an unverified email.	**MUST**

WDR-10	Only balance from approved deposits and refunds is withdrawable. Funds that have never been used for a purchase can be withdrawn only after a configurable holding period (default 24h) to reduce money-laundering and chargeback-style abuse.	**SHOULD**

6.6 Ledger (Source of Truth)

ID	Requirement	Priority
LED-01	Every balance change creates one immutable `WalletTransaction` row. Rows are never updated or deleted. Corrections are made by new compensating rows.	**MUST**
LED-02	Each row stores: type, amount (always positive), direction (CREDIT/DEBIT), `balanceBefore`, `balanceAfter`, reference type/ID (deposit, withdrawal, order), actor (user or admin), reason and timestamp.	**MUST**
LED-03	Entry types: `DEPOSIT`, `ORDER_PAYMENT`, `REFUND`, `WITHDRAWAL_HOLD`, `WITHDRAWAL_RELEASE`, `WITHDRAWAL`, `FEE`, `ADMIN_CREDIT`, `ADMIN_DEBIT`, `REVERSAL`.	**MUST**
LED-04	Invariant: `wallet.balance = Σ credits − Σ debits`. A nightly job recomputes this for every wallet and alerts on any mismatch.	**MUST**
LED-05	The customer's History page lists ledger entries and request statuses with filters (type, date range, status), pagination and CSV export.	**MUST**
LED-06	Every wallet mutation is written to AuditLog with actor, action, target and payload (extends FR-29).	**MUST**

6.7 Admin Panel

ID	Requirement	Priority
ADM-01	Dashboard KPI cards: Total Users, Total Wallet Liability (sum of all balances), Pending Deposits (count + amount), Pending Withdrawals (count + amount), Today's Approved Deposits, Today's Paid Withdrawals.	**MUST**
ADM-02	Tabs: Deposits · Withdrawals · Users · Orders/Refunds · Settings · Reports.	**MUST**
ADM-03	Deposits tab: filter by status/method/date/user/TrxID search; row actions Approve / Decline (with reason modal); badges Pending, Approved, Declined, Stale; bulk-approve disabled by design to force a check on every TrxID.	**MUST**
ADM-04	Withdrawals tab: Approve, Reject, Mark as Paid (with outgoing TrxID).	**MUST**
ADM-05	Users tab: balance, total deposited, total spent, total withdrawn, status, freeze/unfreeze, ledger view, manual adjustment (reason required, per-action cap, second-admin confirmation above a configurable threshold).	**MUST**
ADM-06	Settings tab: per-method on/off, receiving account/number/address, QR image, account type, min/max, fee, USDT→BDT rate and its last-updated time, daily limits, stale-deposit hours, refund policy text, maintenance mode for the wallet.	**MUST**
ADM-07	Reports: daily/monthly deposits, withdrawals, net inflow, per-method totals, reconciliation status, CSV export.	**SHOULD**
ADM-08	Real-time or 30-second polling refresh of pending counts and a sound/visual alert for new deposits.	**COULD**

6.8 Notifications

Event	Recipient	Channel
Deposit submitted	Customer, Admins	In-app (+ admin badge)
Deposit approved / declined (with reason)	Customer	In-app
Order paid from wallet	Customer	In-app
Refund credited	Customer	In-app
Withdrawal submitted / approved / paid / rejected	Customer (Admin on submit)	In-app
Low-balance warning when pending order needs top-up	Customer	In-app
Reconciliation mismatch	Admins	In-app

7. Core Flows

7.1 Deposit

1. Customer opens Deposit, picks a method, copies the account number.
2. Sends money from the external app.
3. Enters amount + TrxID (+ optional screenshot) and presses Confirm.
4. Server validates limits and uniqueness and creates the DepositRequest as PENDING.
5. Admin verifies the money in the real bKash/Nagad/Binance account, then Approves or Declines.
6. On Approve, inside one transaction: lock wallet row → write DEPOSIT ledger entry → update balance → mark deposit APPROVED → write audit log → create notification.
7. Customer sees the new balance and the History entry.

7.2 Checkout with Wallet

1. Customer reviews the cart and chooses the founder (unchanged from v1.0).
2. Checkout shows total and wallet balance.
3. Sufficient: Pay with Wallet → order created SUBMITTED + payment PAID + ledger debit, atomically.
4. Insufficient: Top up shortfall → deposit approved → return to checkout.
5. The order then follows the 7-stage pipeline.

7.3 Withdrawal

1. Customer opens Withdraw, chooses method, destination and amount.
2. System holds the amount (WITHDRAWAL_HOLD).
3. Admin sends the money externally and marks Paid with the outgoing TrxID.
4. Ledger finalizes with WITHDRAWAL; customer is notified. If rejected, the hold is released.

7.4 Cancellation Refund

Cancel/reject → server checks amountPaid → REFUND ledger entry → wallet credited → notification.

8. Order Lifecycle Impact

The 7-stage pipeline is unchanged. Payment now interacts with it as follows:

Stage	Payment rule in v1.1
SUBMITTED	Order exists only if payment succeeded. No unpaid orders are created.
UNDER_REVIEW	Funds are held as PAID. Customer cancel → full refund to wallet.
APPROVED → IN_PROGRESS	No payment step needed (previously "payment being confirmed"). Founder may start immediately.
QUALITY_REVIEW / COMPLETED / DELIVERED	No payment action. (With PAY-08 enabled, the final instalment is charged at COMPLETED.)
REJECTED / CANCELLED	Automatic refund of the net paid amount.

Result: the old gating rule "payment status gates order progression" (FR-17) is satisfied at order creation, which removes the payment-verification wait from the delivery timeline.

9. Data Model Changes

UUID primary keys, `Decimal(12,2)` money, all timestamps in UTC.

Model	Key fields	Notes
Wallet (new)	id, userId (unique), balance, heldBalance, status, frozenReason, version	`CHECK (balance >= 0 AND heldBalance >= 0)`; version for optimistic locking
WalletTransaction (new)	id, walletId, type, direction, amount, balanceBefore, balanceAfter, refType, refId, actorId, reason, createdAt	Append-only; indexed on `(walletId, createdAt)` and `(refType, refId)`
DepositRequest (new)	id, userId, method, amountClaimed, amountCredited, currency, usdtAmount, fxRate, trxId, trxIdNormalized, screenshotUrl, status, declineReason, adminNote, reviewedBy, reviewedAt, createdAt	Unique index on `(method, trxIdNormalized)` where status ≠ DECLINED
WithdrawalRequest (new)	id, userId, method, destination, network, amount, fee, status, rejectReason, outgoingTrxId, proofUrl, reviewedBy, paidAt, createdAt	Hold ledger entry referenced by refId
PaymentMethodConfig (new)	method (PK), enabled, accountLabel, accountValue, accountType, qrUrl, minDeposit, maxDeposit, minWithdraw, maxWithdraw, feePercent, feeFlat, updatedBy, updatedAt	Editable from admin Settings
WalletSetting (new)	key, value (JSON)	USDT→BDT rate, stale hours, daily caps, refund policy text, maintenance flag
Payment (changed)	add `provider = WALLET`, walletTransactionId, amountPaid, amountRefunded	Replaces manual TrxID submission for orders; old rows kept as `provider = MANUAL`
Order (changed)	add amountPaid, amountRefunded, paymentStatus (`PAID`, `PARTIALLY_REFUNDED`, `REFUNDED`)	Order creation requires a successful wallet debit
AuditLog	unchanged structure	New action names for all wallet events
Notification	new types	Wallet event types

Required database guarantees

- All balance changes run in a serializable transaction or use `SELECT … FOR UPDATE` on the wallet row to prevent double-spend under concurrent requests.

- Unique constraint on `(method, normalized TrxID)` to stop duplicate credits.
- `CHECK` constraints for non-negative balances.
- Ledger tables have no UPDATE/DELETE privileges for the application role.

10. API Changes (Express, under `/api`, uniform envelope)

Group	Endpoint	Auth
Wallet	`GET /wallet` (balance, held, status) · `GET /wallet/transactions` (filters, pagination) · `GET /wallet/transactions/export` (CSV)	Authenticated
Payment methods	`GET /payment-methods` (enabled methods, receiving account, limits, fees, rate)	Authenticated
Deposits	`POST /deposits` · `GET /deposits` · `GET /deposits/:id` · `POST /deposits/:id/cancel`	Authenticated + ownership
Withdrawals	`POST /withdrawals` · `GET /withdrawals` · `POST /withdrawals/:id/cancel`	Authenticated + ownership
Orders	`POST /orders` now takes `paymentMode: "WALLET"` and the idempotency key; returns `INSUFFICIENT_BALANCE` with the shortfall when needed. `POST /orders/:id/payments` is deprecated (kept read-only for legacy orders).	Authenticated
Admin – deposits	`GET /admin/deposits` · `POST /admin/deposits/:id/approve` · `POST /admin/deposits/:id/decline`	ADMIN (audited)
Admin – withdrawals	`GET /admin/withdrawals` · `POST /admin/withdrawals/:id/approve` · `…/reject` · `…/mark-paid`	ADMIN (audited)
Admin – wallets	`GET /admin/wallets` · `GET /admin/wallets/:userId` · `POST /admin/wallets/:userId/adjust` · `POST /admin/wallets/:userId/freeze`	ADMIN (audited)
Admin – refunds	`POST /admin/orders/:id/refund`	ADMIN (audited)
Admin – settings	`GET/PUT /admin/payment-methods` · `GET/PUT /admin/wallet-settings`	ADMIN (audited)
Admin – reports	`GET /admin/reports/wallet` · `GET /admin/reports/reconciliation`	ADMIN

New error codes: `INSUFFICIENT_BALANCE`, `DUPLICATE_TRX_ID`, `AMOUNT_OUT_OF_RANGE`, `METHOD_DISABLED`, `WALLET_FROZEN`, `WITHDRAWAL_BLOCKED`, `ALREADY_PROCESSED`, `RATE_LIMITED`, `INVALID_DESTINATION`.

Every request body is zod-validated. Money is always sent and returned as a string.

11. Page Map Changes

Area	Route	Purpose

Customer	`/wallet`	Overview: balance, held amount, quick Deposit/Withdraw, recent activity
Customer	`/wallet/deposit`	Method tabs, copy-account box, steps, amount + TrxID + screenshot, status of pending deposits
Customer	`/wallet/withdraw`	Method, destination, amount, fee preview, refund policy
Customer	`/wallet/history`	Ledger and request history with filters and CSV export
Checkout	`/checkout`	Wallet balance panel, Pay with Wallet, Top-up shortcut
Customer	`/orders/[id]`	Payment section shows PAID, receipt, refund status (no TrxID form)
Admin	`/admin`	New KPI cards
Admin	`/admin/deposits`, `/admin/withdrawals`	Queues with actions
Admin	`/admin/wallets`, `/admin/wallets/[userId]`	User balances, ledger, adjust, freeze
Admin	`/admin/settings/payments`	Methods, limits, fees, rate, policy
Admin	`/admin/reports`	Financial and reconciliation reports

The existing `/admin/payments` page is retired and replaced by deposits/withdrawals.

12. UX Requirements

- Keep the visual language of the reference (dark green theme, yellow primary CTA, rounded cards, Bengali labels). Support Bangla and English with a language toggle; all money shown as ৳ 1,234.50.
- Deposit page: selected method highlighted, copy button gives feedback, inline validation messages, submit button disabled until valid, and a clear "waiting for admin approval" state with an estimated time.
- Status badges with consistent colors: **Pending** (amber), **Approved/Paid** (green), **Declined/Rejected** (red), **Stale** (orange), **Held** (blue).
- Confirmation modal before any irreversible admin action; decline requires a reason.
- Empty, loading and error states for every list; mobile-first layout (Bangladesh traffic is mostly mobile) with a bottom-sheet pattern for forms.
- Accessibility: keyboard navigable, minimum 4.5:1 contrast, status shown by text as well as color.

13. Security, Fraud & Compliance

Risk	Control
Fake or reused TrxID	Unique normalized TrxID per method; admin must verify in the real merchant account before approving; screenshot required above a threshold
Double-spend / race conditions	Row locks or serializable transactions; idempotency keys; version column
Admin abuse or error	AuditLog on every action; manual adjustment needs a reason, a per-action cap and second-admin approval over a threshold; admins cannot spend customer balances; dual-control for changing receiving account numbers
Account takeover → withdrawal	Re-authentication, 24h cooling for new destinations, email notification on every withdrawal, daily caps
Money laundering / chargeback-style abuse	Holding period before withdrawing unused deposits, velocity limits, freeze capability, flagging of repeated deposit-then-withdraw behaviour
Receiving-account phishing	Receiving numbers come only from authenticated server settings, never from client input
Data exposure	Customers see only their own wallet; screenshots served by signed Cloudinary URLs; no stack traces in errors
Rate abuse	Per-user and per-IP limits on deposit/withdrawal endpoints; stricter limiter retained for auth/contact

Regulatory	Holding customer funds as stored value may require legal review in Bangladesh (e-money / payment service provider rules, Bangladesh Bank, and rules on crypto such as Binance). Legal and tax advice must be obtained before launch; this PRD is not legal advice. Keep wallet use limited to TSH services only (closed-loop).

14. Non-Functional Requirements

- **Consistency:** zero tolerance for balance drift; nightly reconciliation job with alerts.
- **Performance:** wallet and history pages < 500 ms p95; checkout payment < 2 s p95.
- **Availability:** wallet maintenance-mode switch; deposits can still be queued when payments are paused.
- **Backups & recovery:** point-in-time recovery on Neon; ledger exportable to CSV daily.
- **Observability:** structured logs with request ID, alerts for stale deposits, reconciliation mismatches, and error-rate spikes.
- **Testing:** Vitest suites for money math, ledger invariants, concurrency (parallel debits), idempotency, state transitions, permission checks, and migration scripts. Target ≥ 90% coverage on wallet domain code.
- **Serverless note:** because Express runs in Vercel functions, all multi-step wallet operations must be single database transactions; no in-memory state or background timers (use scheduled Vercel Cron for reconciliation and stale-deposit flags).

15. Migration Plan

1. **Schema:** add new tables and columns (additive only; no destructive change).
2. **Back-fill:** create zero-balance wallets for all users; mark existing payments `provider = MANUAL`.
3. **Open orders in flight:** orders with PENDING manual payments continue through the old verification path (read-only admin queue) until resolved; new orders must use the wallet.
4. **Feature flag:** `WALLET_ENABLED` per environment. Roll out to staging, then to the two founders' accounts, then to all users.
5. **Data check:** compare totals before/after; run reconciliation; keep the old payments page available read-only for 30 days.
6. **Cut-over:** disable `POST /orders/:id/payments` for new orders; announce to customers in-app.
7. **Rollback:** feature flag off restores the legacy flow; wallet balances remain intact and can be refunded manually via the withdrawal process.

16. Delivery Phases

Phase	Scope	Exit criteria
1 — Foundations	Schema, wallet creation, ledger service, settings, payment-method config	Ledger invariant tests pass
2 — Deposits	Customer deposit page, admin deposit queue, notifications	End-to-end approve/decline works; duplicate TrxID blocked
3 — Wallet checkout	Pay with Wallet, top-up shortcut, order page changes, refunds	Atomic order + debit proven under concurrency tests
4 — Withdrawals	Withdraw page, admin queue, holds, mark-paid	Hold/release/finalize reconciles to zero
5 — Admin & reports	Users tab, adjustments, freeze, reports, reconciliation job	Nightly reconciliation green for 7 days
6 — Hardening & launch	Security review, load test, legal check, migration, rollout	All acceptance criteria met

17. Acceptance Criteria (Key Scenarios)

1. A customer submitting the same TrxID twice for the same method gets `DUPLICATE_TRX_ID` and no second request is created.
2. Approving a ৳1,500 deposit increases the balance by exactly ৳1,500.00, creates one ledger row with correct before/after balances, and cannot be approved a second time.
3. Declining a deposit requires a reason, leaves the balance unchanged, and the customer sees the reason.

4. Two simultaneous checkouts that together exceed the balance: exactly one succeeds, the other returns `INSUFFICIENT_BALANCE`.
5. Retrying the same checkout request with the same idempotency key charges once.
6. Cancelling an order in UNDER_REVIEW refunds the full paid amount to the wallet automatically.
7. A withdrawal request holds funds immediately; rejection releases them; marking paid finalizes with an outgoing TrxID.
8. A frozen wallet cannot pay or withdraw but can view history.
9. A customer cannot read or modify another user's wallet, deposits, or withdrawals (403/404).
10. The nightly reconciliation job reports zero mismatches on the seed and test datasets.
11. Every admin action appears in the AuditLog with actor, target and payload.
12. The Binance deposit stores the USDT amount, the exchange rate used, and the credited BDT amount.

18. Analytics & KPIs to Instrument

Deposit funnel (opened → submitted → approved), median approval time, decline rate by reason, average deposit size, wallet-funded checkout rate, top-up-from-checkout conversion, withdrawal rate and average time to pay out, total wallet liability over time, and reconciliation mismatches.

19. Open Questions for Founders

#	Question	Recommended default
1	Should unused balance be withdrawable in full, or only refunds?	Allow withdrawal of unused balance after a 24h holding period
2	Any fee on deposits or withdrawals?	0% at launch; fields exist in Settings
3	Binance: USDT only, which network (e.g. TRC20/BEP20), and who sets the USDT→BDT rate?	USDT-TRC20, rate set manually by admin and shown with timestamp
4	Add Rocket and bank transfer as in v1.0?	Add Rocket now; bank transfer later
5	Which founder(s) may approve deposits, and is a second approver needed above a threshold?	Either founder; second approver for adjustments above ৳10,000
6	Enable partial/milestone payments (PAY-08)?	Not in v1.1
7	Legal review of holding customer funds and accepting crypto in Bangladesh	Required before public launch
8	Minimum/maximum amounts and daily limits	Deposit ৳100–৳50,000; withdrawal ৳200–৳50,000; daily cap ৳100,000

20. Changes to PRD v1.0 (Traceability)

v1.0 item	v1.1 status
FR-16 Manual TrxID submission per order	Replaced by DEP-01 to DEP-15 and PAY-01 to PAY-07
FR-17 Admin verifies each order payment	Replaced by deposit approval (DEP-07 to DEP-10)
FR-18 Gateway-ready schema	Kept, extended with the provider field and WALLET provider
FR-24 Notifications	Extended with wallet events (6.8)
FR-26 Admin dashboard stats	Extended with wallet KPIs (ADM-01)
FR-29 AuditLog	Extended to all wallet mutations (LED-06)
Section 4 "Out of scope: automated gateways"	Unchanged; still post-v1.1
Section 6 pipeline	Unchanged; payment gating moved to order creation
Section 12 Known Gaps	Add: legal review, wallet reconciliation job, automated gateway integration

Status: Proposed — pending founder sign-off. Section 19 open questions must be answered before Phase 1 begins; the regulatory item (Section 13, row 9) requires external legal advice before public launch.